

LABORATÓRIO TPSE I



Prática 04: Programando Funcionalidade Timers

Prof. Francisco Helder

27 de maio de 2026

Sumário

1	Timer	2
1.1	Descrição da Funcionalidade	3
1.1.1	Funcionalidade Modo Temporizador	3
1.2	Funcionalidade Modo de Captura	6
1.3	Funcionalidade Modo Comparação	6
1.4	Funcionalidade Prescaler	6
1.5	Taxa de contagem do Timer	7
2	Atividades Práticas	8

1 Timer

O módulo Timer contém um contador ascendente com capacidade de recarga automática em caso de overflow. O contador do Timer pode ser lido e escrito em tempo real (durante a contagem). O módulo Timer inclui lógica de comparação para permitir um evento de interrupção em um valor de contador programável.

Um sinal de saída dedicado pode ser pulsado ou alternado no evento inicial e de overflow. Esta saída oferece um sinal de disparo de tempo ou sinal PWM (modulação de largura de pulso). Um sinal de saída dedicado pode ser usado para uso geral (diretamente acionado pelo bit 14 do registrador TCLR). Um sinal de entrada dedicado é usado para acionar a captura automática do contador do Timer e o evento de interrupção, no tipo de transição de sinal de entrada programável. Um divisor de clock programável (prescaler) permite a redução da frequência do clock de entrada do Timer. Todas as fontes de interrupção do Timer interno são mescladas em uma linha de interrupção de módulo e uma linha de ativação. Cada fonte de interrupção interna pode ser habilitada/desabilitada independentemente.

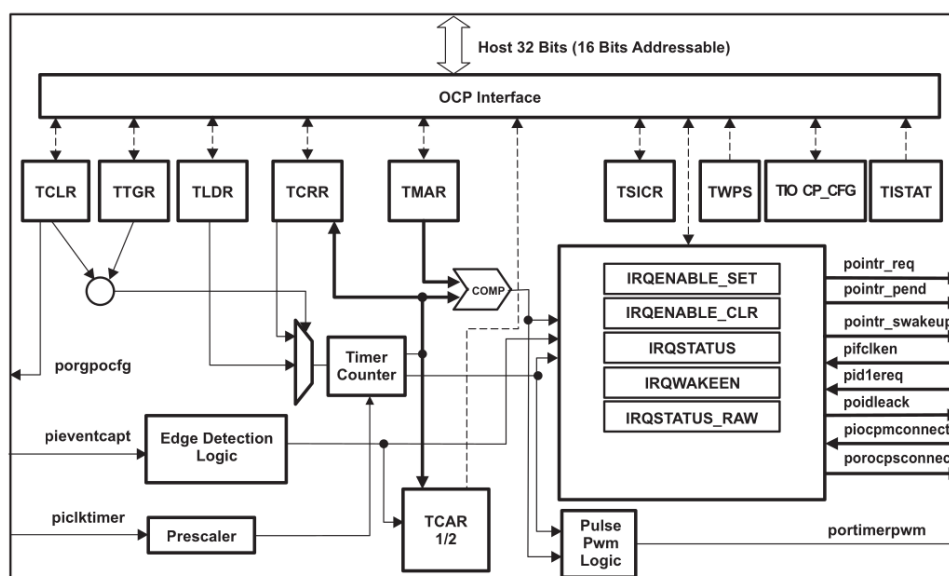


Figura 1: A figura mostra um diagrama de blocos do Timer.

Este módulo é controlável através do barramento periférico OCP, como visto na Figura 1. Como dois domínios de clock são gerenciados dentro deste módulo, a resincronização é feita por lógica especial entre o domínio de clock OCP e o domínio de clock Timer. Na reinicialização, a lógica de sincronização permite a utilização de todas as relações entre o clock OCP e o clock Timer. Uma desvantagem deste modo é que o caminho de resincronização completa é usado com impacto no desempenho da latência de acesso em termos de ciclos de clock OCP.

Para melhorar a latência de acesso do módulo, e sob condições restritas em relações de clocks, o modo write-posted pode ser usado definindo o bit POSTED do System Control Register (TSCR). Neste modo, o modo write posted é habilitado, o que significa que o comando de gravação OCP é concedido antes que o processo de gravação seja concluído no domínio de clock timer. Este modo permite que o software faça gravações simultâneas em registros de timer de Modo Dual e observe a conclusão do processo de gravação (sincronização) no nível do software lendo bits de status de gravação postados independentes no Write Posted Status Register (TWPS).

1.1 Descrição da Funcionalidade

O Timer de uso geral é um contador ascendente. Suporta 3 modos funcionais:

- Modo temporizador
- Modo de captura
- Modo de comparação

Por padrão, após a reinicialização do núcleo, os modos de captura e comparação são desabilitados.

1.1.1 Funcionalidade Modo Temporizador

O Timer é um contador ascendente que pode ser iniciado e parado a qualquer momento através do Timer Control Register (bit TCLR ST). O Registro do Contador de Temporizador (TCRR) pode ser carregado quando parado ou em tempo real (durante a contagem). O TCRR pode ser carregado diretamente por um acesso de gravação TCRR com o novo valor do temporizador. O TCRR também pode ser carregado com o valor mantido no Timer Load Register (TLDR) por um trigger register (TTGR) Acesso de gravação. O carregamento do TCRR é feito independentemente do valor gravado no TTGR. O valor do registro do contador do temporizador (TCRR) pode ser lido quando parado ou capturado em tempo real por um acesso de leitura no TCRR. O Timer é parado e o valor do contador é limpo para “0” quando o reset do módulo é afirmado. O Timer é mantido parado após o reset ser liberado. Quando o temporizador é parado, o TCRR é congelado, e o cronômetro pode ser reiniciado a partir do valor congelado, a menos que o TCRR tenha sido recarregado com um novo valor (veja Figura 2).

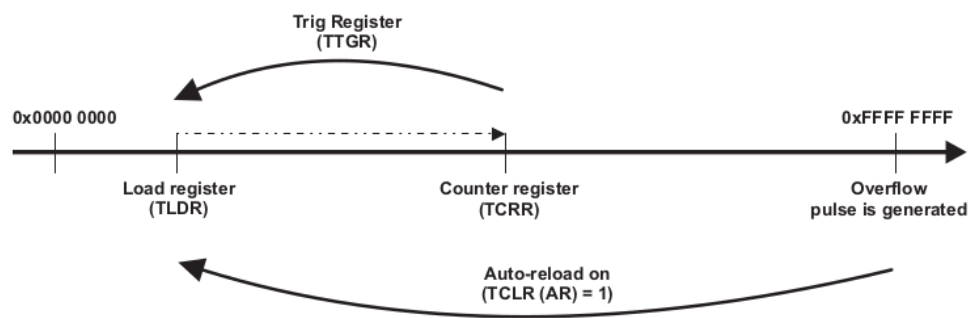


Figura 2: Valor do tempo no registrador TCRR.

No modo one shot (bit TCLR AR = 0), o contador é parado após o overflow da contagem (o valor do contador permanece em zero). Quando o modo de auto-recarregamento está habilitado (TCLR AR bit = 1), o TCRR é recarregado com o valor do Timer Load Register (TLDR) após um estouro de contagem.

NOTE: não é recomendado colocar o valor de overflow (FFFF FFFFh) no TLDR porque pode levar a resultados indesejados.

Uma interrupção pode ser usada em overflow se o bit de habilitação de interrupção de overflow estiver definido no registro de interrupção do Timer (bit IRQENABLE_SET OVF_IT_FLAG = 1). Um pino de saída dedicado (PORTIMERPWM) é programado através de TCLR (bits TRG

e PT) para gerar um pulso positivo (duração do prescaler) ou para inverter o valor atual (modo de alternância) quando ocorre um overflow.

No código abaixo temos a configuração inicial do Timer, nesse caso será utilizado o módulo DMTIMER7, que deve ser criada no início do sistema:

```

1 /* Clock enable for DMTIMER7 TRM 8.1.12.1.25 */
2 ckmSetCLKModuleRegister(CKM_PER_TIMER7_CLKCTRL,
   CM_PER_TIMER7_CLKCTRL_MODULEMODE_ENABLE);
3
4 while ((HWREG(SOC_CM_PER_REGS + CKM_PER_TIMER7_CLKCTRL) &
5 CM_PER_TIMER7_CLKCTRL_MODULEMODE) !=
   CM_PER_TIMER7_CLKCTRL_MODULEMODE_ENABLE) {};
6
7 // Configures the interface to Posted mode for better write latency
8 // Bit 2 (POSTED) enabled enables posted mode.
9 HWREG(SOC_DMTIMER_7_REGS+DMTIMER_TSICR) |= DMTIMER_TSICR_POSTED;
10
11 //Ensures the timer is stopped before configuration
12 while (HWREG(SOC_DMTIMER_7_REGS+DMTIMER_TWPS) &
   DMTIMER_TWPS_W_PEND_TCLR); // Waits for past synchronizations
13 HWREG(SOC_DMTIMER_7_REGS+DMTIMER_TCLR) &= ~DMTIMER_TCLR_ST;
14
15 //Resets the counter and load register
16 while (HWREG(SOC_DMTIMER_7_REGS+DMTIMER_TWPS) &
   DMTIMER_TWPS_W_PEND_TCRR);
17
18 HWREG(SOC_DMTIMER_7_REGS+DMTIMER_TCRR) = 0x00000000;
19 HWREG(SOC_DMTIMER_7_REGS+DMTIMER_TLDR) = 0x00000000;

```

A Tabela abaixo apresenta os principais registradores utilizados para a configuração e controle do módulo DMTIMER7:

Tabela 1: Registradores importante para uso do timer com contador.

Registrador	Endereço	Função
CM_PER_TIMER7_CLKCTRL	0x44E0007C	Habilita o clock para o Timer7.
TCLR (Timer Control)	0x4804A024	Controla o início/fim e modo do timer.
TCRR (Timer Counter)	0x4804A028	O contador (incrementa a cada ciclo).
TLDR (Timer Load)	0x4804A02C	Registrador de recarga .
TWPS (Timer status)	0x4804A048	Usado para ler status do Timer.

Já para calcular o tempo de atraso (ou aguardo), pressupondo um clock funcional (Func Clock) configurado para 24 MHz e o pré-escalador desativado 24 MHz significa 24.000.000 incrementos por segundo = 24 incrementos por microssegundo. No código abaixo temos um exemplo de função de Delay e suas funções dependentes:

```

1 static unsigned int DMTimerWritePostedStatusGet(unsigned int baseAdd)
2 {
3     /* Return the status of TWPS register */
4     return (HWREG(baseAdd + DMTIMER_TWPS));
5 }

```

```
6 #define DMTimerWaitForWrite(reg, baseAdd) \
7 if(HWREG(baseAdd + DMTIMER_TSICR) & DMTIMER_TSICR_POSTED) \
8 while((reg & DMTimerWritePostedStatusGet(baseAdd)));
9
10 void DMTimerCounterSet(unsigned int baseAdd, unsigned int counter) {
11     /* Wait for previous write to complete */
12     DMTimerWaitForWrite(DMTIMER_WRITE_POST_TCRR, baseAdd);
13
14     /* Set the counter value */
15     HWREG(baseAdd + DMTIMER_TCRR) = counter;
16 }
17
18 unsigned int DMTimerCounterGet(unsigned int baseAdd) {
19     /* Wait for previous write to complete */
20     DMTimerWaitForWrite(DMTIMER_WRITE_POST_TCRR, baseAdd);
21
22     /* Read the counter value from TCRR */
23     return (HWREG(baseAdd + DMTIMER_TCRR));
24 }
25
26 void DMTimerEnable(unsigned int baseAdd) {
27
28     /* Wait for previous write to complete */
29     DMTimerWaitForWrite(DMTIMER_WRITE_POST_TCLR, baseAdd);
30
31     /* Start the timer */
32     HWREG(baseAdd + DMTIMER_TCLR) |= DMTIMER_TCLR_ST;
33 }
34
35 void DMTimerDisable(unsigned int baseAdd) {
36
37     /* Wait for previous write to complete */
38     DMTimerWaitForWrite(DMTIMER_WRITE_POST_TCLR, baseAdd);
39
40     /* Stop the timer */
41     HWREG(baseAdd + DMTIMER_TCLR) &= ~DMTIMER_TCLR_ST;
42 }
43
44 void uDelay (unsigned int us) {
45     // Calculates the total number of "ticks" needed
46     unsigned int ticks_needed = us * TIMER_1US_COUNT;
47
48     DMTimerCounterSet(SOC_DMTIMER_7_REGS, 0);
49     DMTimerEnable(SOC_DMTIMER_7_REGS);
50     while(DMTimerCounterGet(SOC_DMTIMER_7_REGS) < ticks_needed);
51     DMTimerDisable(SOC_DMTIMER_7_REGS);
52 } /*
```

1.2 Funcionalidade Modo de Captura

O valor do timer em TCRR pode ser capturado e salvo na função TCAR1 ou TCAR2 do modo selecionado em TCLR através do campo CAPT_MODE quando uma transição é detectada no pino de entrada do módulo (PIEVENTCAPT). O circuito de detecção de borda monitora as transições no pino de entrada (PIEVENTCAPT). Transição ascendente, transição descendente ou ambas podem ser selecionadas em TCLR (bit TCM) para disparar a captura do contador do timer. O módulo define o IRQSTATUS (bit TCAR_IT_FLAG) quando uma transição ativa é detectada e, ao mesmo tempo, o valor do contador TCRR é armazenado em um dos registradores de captura do temporizador TCAR1 ou TCAR2, como segue:

- Se o campo CAPT_MODE do TCLR for 0, então, no primeiro evento de captura habilitado, o valor do registrador do contador é salvo no registrador TCAR1 e todos os próximos eventos são ignorados (sem atualização no TCAR1 e sem acionamento de interrupção) até que a lógica de detecção seja reiniciada ou o registrador de status de interrupção seja limpo na posição do TCAR, escrevendo um 1 nele.
- Se o campo CAPT_MODE do TCLR for 1, então, no primeiro evento capturado habilitado, o valor do contador é salvo no registrador TCAR1 e, no segundo evento de captura habilitado, o valor do registrador do contador é salvo no registrador TCAR2. Todos os outros eventos são ignorados (nenhuma atualização em TCAR1/2 e nenhum disparo de interrupção) até que a lógica de detecção seja reiniciada ou o registro de status de interrupção seja limpo na posição do TCAR, escrevendo um 1 nele. Este mecanismo é útil para o cálculo do período de um relógio se esse relógio estiver conectado ao pino de entrada PIEVENTCAPT.

A lógica de detecção de borda é reiniciada (uma nova captura é habilitada) quando a interrupção de captura ativa é servida. O bit TCAR_IT_FLAG de IRQSTATUS (anteriormente 1) é limpo. O relógio funcional do timer (entrada para o prescaler) é usado para amostrar o pino de entrada (PIEVENTCAPT). Pulsos negativos ou positivos podem ser detectados quando o tempo do pulso excede o período do relógio funcional. Uma interrupção pode ser emitida na detecção de transição se o bit de habilitação de interrupção de captura estiver definido no Registro de habilitação de interrupção do timer IRQENABLE_SET (bit TCAR_IT_FLAG).

1.3 Funcionalidade Modo Comparação

Quando Compare Enable TCLR (bit CE) é definido como 1, o valor do timer (TCRR) é permanentemente comparado ao valor mantido no registro de correspondência do timer (TMAR). O valor TMAR pode ser carregado a qualquer momento (contagem ou parada do timer). Quando os valores TCRR e TMAR correspondem, uma interrupção pode ser emitida se o IRQENABLE_SET (bit MAT_EN_FLAG) estiver definido. A implementação correta é escrever um valor de comparação no registro TMAR antes de definir TCLR (bit CE) para evitar interrupções indesejadas devido a um efeito de correspondência de valor de redefinição.

O pino de saída dedicado (PORTIMERPWM) pode ser programado por meio de TCLR (bits TRG e PT) para gerar um pulso positivo (duração do relógio TIMER) ou para inverter o valor atual (modo de alternância) quando ocorrer um estouro e uma correspondência.

1.4 Funcionalidade Prescaler

Um contador prescaler pode ser usado para dividir a frequência do clock de entrada do contador do timer. O prescaler é habilitado quando o bit 5 do TCLR é definido (PRE). O valor

da razão de divisão 2^n (PTV) pode ser configurado no registro TCLR, então o contador prescaler é reiniciado quando o contador do timer é parado ou recarregado em tempo real.

1.5 Taxa de contagem do Timer

O contador do timer é composto de um estágio prescaler e um contador do timer. O estágio prescaler é sincronizado com o clock do timer e atua como um divisor de clock para o estágio do contador do timer. A taxa pode ser gerenciada acessando o campo de definição de taxa do registro de controle (PTV e PRE do TCLR), como visto na Tabela 2.

Pre	PTV	Divisor (PS)
0	X	1
1	0	2
1	1	4
1	2	8
1	3	16
1	4	32
1	5	64
1	6	128
1	7	256

Tabela 2: Valor das relações de clock do prescaler.

A taxa do Timer é definida por:

- O valor dos campos do prescaler (PRE e PTV do registro TCLR);
- O valor carregado no Timer Load Register (TLDR).

A equação da taxa do timer é a seguinte:

$$(FFFF\ FFFFh - TLDR + 1) \times \text{timer Clock period} \times \text{Clock Divider (PS)}$$

$$\text{With timer Clock period} = 1/(\text{timerClockfrequency}) \text{ and } PS = 2^{(PTV+1)}.$$

Como exemplo, se considerarmos uma entrada de relógio temporizador de 32 kHz, com um campo PRE igual a 0, o período de saída do timer é:

TLDR	Interrupt period
0000 0000h	37 h
FFFF 0000h	2 s
FFFF FFF0h	500 us
FFFF FF FEh	62,5 us

Tabela 3: Valor e Período de Interrupção Correspondente.

Para mais detalhes sobre Timer ler o capítulo “20. Timer” do datasheet.

2 Atividades Práticas

pratica 1:

Recrie a função delay com valores preciso de tempo, então faça os leds piscarem com frequências definidas e mostre essa frequência no osciloscópio.

pratica 2:

Configure o UART e crie um menu no terminal para escolher frequências diferentes.